



HOJA DE AYUDA – EXAMEN FRONTEND · IISSI2-IS-2026

Editable. Añade/borra lo que necesites. Basada en Labs 4–8 y
Trial-Exam-Frontend.

0. ARRANQUE RÁPIDO DEL PROYECTO

Instalar dependencias (hacer solo una vez)

`npm run install:all:bash` # Linux/Mac

`npm run install:all:win` # Windows

Recrear BD con datos de prueba

`npm run migrate:backend`

Arrancar backend (terminal 1)

`npm run start:backend`

Arrancar frontend (terminal 2)

`npm run start:frontend`

Credenciales de prueba del seeder:

- Email: `owner1@owner.com`
- Password: `secret`

1. ESTRUCTURA DE FICHEROS CLAVE

DeliverUS-Frontend-Owner/src/

|— api/

- | | — helpers/ApiRequestsHelper.js ← get, post, put, patch, destroy
- | | — RestaurantEndpoints.js ← funciones de API para restaurantes
- | | — OrderEndpoints.js ← funciones de API para pedidos
- | | — AuthEndpoints.js
- | — screens/
 - | | — restaurants/
 - | | | — RestaurantsScreen.js
 - | | | — RestaurantDetailScreen.js
 - | | | — CreateRestaurantScreen.js
 - | | | — EditRestaurantScreen.js
 - | | — orders/ ← EXAMEN: aquí trabajas
 - | | | — OrdersScreen.js
 - | | | — EditOrderScreen.js
- | — navigation/
 - | — RestaurantsStack.js ← registrar pantallas nuevas aquí

2. PATRÓN useEffect + FETCH (Lab 5)

Plantilla estándar. **Define el fetch FUERA del useEffect** para poder reutilizarlo (ej: refrescar tras un cambio de estado o un delete):

```
import { useContext, useEffect, useState } from 'react'
```

```
import { AuthorizationContext } from '../../../../context/AuthorizationContext'
```

```
import { showMessage } from 'react-native-flash-message'
```

// Dentro del componente:

```
const { loggedInUser } = useContext(AuthorizationContext)
```

```
const [items, setItems] = useState([])
```

// fetch definido fuera del useEffect → reutilizable en handleNextStatus, removeItem, etc.

```
const fetchItems = async () => {
```

```
  try {
```

```
    const fetched = await getItemsFromAPI() // función del Endpoint
```

```
    setItems(fetched)
```

```
  } catch (error) {
```

```
    showMessage({
```

```
      message: `Error al cargar datos. ${error}`,
```

```
      type: 'error',
```

```
      style: GlobalStyles.flashStyle,
```

```
      titleStyle: GlobalStyles.flashTextStyle
```

```
    })
```

```
  }
```

```
}
```

```
useEffect(() => {
```

```
  if (loggedInUser) {
```

```
    fetchItems()
```

```
  } else {
```

```
    setItems(null)

  }

}, [loggedInUser, route]) // añadir 'route' para refrescar al navegar
```

IMPORTANTE: Si los datos son públicos (sin login), omite el `if (loggedInUser)` y pon el trigger que necesites.

3. FUNCIONES DE API (Endpoints)

Los verbos vienen de ApiRequestsHelper (no los inventas tú). Las rutas del backend te dicen a qué URL llamar y qué verbo HTTP usa, y tú mapeas así:

Verbo HTTP (backend)	Función en ApiRequestsHelper
GET	<code>get(ruta)</code>
POST	<code>post(ruta, data)</code>
PUT	<code>put(ruta, data)</code>
PATCH	<code>patch(ruta, data)</code>
DELETE	<code>destroy(ruta)</code>

// RestaurantEndpoints.js — importar según necesidad:

```
import { get, post, put, patch, destroy } from './helpers/ApiRequestsHelper'
```

// GET todos mis restaurantes

```
function getAll() { return get('/users/myrestaurants') }
```

// GET detalle de un restaurante

```
function getDetail(id) { return get(`restaurants/${id}`) }
```

// GET pedidos de un restaurante ← EXAMEN Trial

```
function getRestaurantOrders(restaurantId) {
```

```
    return get(`restaurants/${restaurantId}/orders`)
  }

// GET analíticas de un restaurante ← EXAMEN Trial

function getRestaurantAnalytics(restaurantId) {

  return get(`restaurants/${restaurantId}/analytics`)

}

// POST crear

function create(data) { return post('restaurants', data) }

// PUT editar

function update(id, data) { return put(`restaurants/${id}`, data) }

// DELETE eliminar

function remove(id) { return destroy(`restaurants/${id}`) }

// PATCH cambiar estado de pedido ← EXAMEN Trial

function nextStatus(order) { return patch(`orders/${order.id}/nextStatus`) }
```

4. NAVEGACIÓN — REGISTRAR PANTALLA NUEVA

En `RestaurantsStack.js` añadir:

```
import OrdersScreen from '../screens/orders/OrdersScreen'

import EditOrderScreen from '../screens/orders/EditOrderScreen'

// Dentro del <Stack.Navigator>:

<Stack.Screen
```

```
    name="OrdersScreen"

    component={OrdersScreen}

    options={{ title: 'Orders' }}

  />

<Stack.Screen

  name="EditOrderScreen"

  component={EditOrderScreen}

  options={{ title: 'Edit Order' }}

/>
```

Navegar a una pantalla pasando parámetros:

// Desde el botón:

```
navigation.navigate('OrdersScreen', { id: item.id })
```

```
navigation.navigate('EditOrderScreen', { id: order.id, restaurantId: restaurantId })
```

// Recoger el parámetro en la pantalla destino:

```
export default function OrdersScreen({ navigation, route }) {

  const restaurantId = route.params.id

}
```

Volver atrás con refresco:

```
navigation.navigate('OrdersScreen', { dirty: true })
```

// El useEffect de OrdersScreen debe tener [route] en su array de dependencias

5. RENDERIZAR LISTAS con FlatList

```
<FlatList
  data={orders}

  keyExtractor={({item}) => item.id.toString()}

  renderItem={({item}) => renderOrder(item)}

  ListEmptyComponent={
    <TextRegular>No hay pedidos para este restaurante.</TextRegular>
  }
/>
```

Función de render de cada ítem:

```
const renderOrder = (item) => {

  return (

    <View style={styles.card}>

      <Image source={getOrderImage(item.status)} style={styles.image} />

      <TextRegular>{item.deliveryAddress}</TextRegular>

      <TextRegular>{item.price} €</TextRegular>

      <TextRegular>{new Date(item.createdAt).toLocaleDateString()}</TextRegular>

      {/* Botón Editar */}

      <Pressable onPress={() => navigation.navigate('EditOrderScreen', { id: item.id, restaurantId:
restaurantId })}

        style={styles.actionButton}>

          <MaterialCommunityIcons name='pencil' color={'white'} size={20} />
```

```

        <TextRegular textStyle={styles.text}>Edit</TextRegular>

    </Pressable>

    {/* Botón Advance estado — solo si no es 'delivered' */}

    {item.status !== 'delivered' &&

        <Pressable onPress={() => handleNextStatus(item)} style={styles.actionButton}>

            <MaterialCommunityIcons name='skip-next' color={'white'} size={20} />

            <TextRegular textStyle={styles.text}>Advance</TextRegular>

        </Pressable>

    }

</View>

)

}

```

6. FORMULARIO CON FORMIK + YUP (Labs 7 y 8)

Esqueleto completo:

```

import { Formik, ErrorMessage } from 'formik'

import * as yup from 'yup'

// 1. Valores iniciales

const initialValues = {

    deliveryAddress: null,

    price: null

```

```
}
```

```
// 2. Schema de validación
```

```
const validationSchema = yup.object().shape({
```

```
  deliveryAddress: yup.string()
```

```
    .max(255, 'Address too long')
```

```
    .required('Delivery address is required'),
```

```
  price: yup.number()
```

```
    .positive('Price must be greater than 0')
```

```
    .required('Price is required')
```

```
})
```

```
// 3. Función de submit
```

```
const handleSubmit = async (values) => {
```

```
  setBackendErrors([])
```

```
  try {
```

```
    const updatedOrder = await update(order.id, values)
```

```
    showMessage({
```

```
      message: `Order updated successfully`,
```

```
      type: 'success',
```

```
      style: GlobalStyles.flashStyle,
```

```
      titleStyle: GlobalStyles.flashTextStyle
```

```
    })
```

```

    navigation.navigate('OrdersScreen', { id: restaurantId, dirty: true })
  } catch (error) {

    setBackendErrors(error.errors)

  }
}

```

// 4. JSX del formulario

```

<Formik

  enableReinitialize

  validationSchema={validationSchema}

  initialValues={initialValues}

  onSubmit={handleSubmit}

>

  ({ handleSubmit, setFieldValue, values }) => (

    <ScrollView>

      <InputItem

        name='deliveryAddress'

        label='Delivery Address'

      />

      <InputItem

        name='price'

        label='Total Price'

        keyboardType='numeric'

```

```

/>

{/* Errores del backend */}

{backendErrors && backendErrors.map((error, index) => (

  <TextError key={index}>{error.param} - {error.msg}</TextError>

))}

{/* Botón de envío */}

<Pressable onPress={handleSubmit} style={styles.button}>

  <TextRegular textStyle={styles.text}>Save</TextRegular>

</Pressable>

</ScrollView>

)}

</Formik>

```

Para formulario de EDICIÓN — cargar datos existentes:

```
// Estado inicial vacío
```

```
const [initialValues, setInitialValues] = useState({ deliveryAddress: null, price: null })
```

```
// useEffect que carga el pedido y rellena el formulario
```

```
useEffect(() => {

  async function fetchOrder() {

    try {

      const fetchedOrder = await getOrder(route.params.id)

      setInitialValues({
```

```

        deliveryAddress: fetchedOrder.deliveryAddress,

        price: fetchedOrder.price

    })

} catch (error) {

    showMessage({ message: `Error: ${error}`, type: 'error', ... })

}

}

fetchOrder()

}, [route])

// enableReinitialize en Formik es OBLIGATORIO para que actualice los valores

```

7. ACCIÓN CAMBIO DE ESTADO (PATCH)

// En OrdersScreen.js

```

const handleNextStatus = async (order) => {

    try {

        await nextStatus(order)      // llamada al endpoint

        await fetchOrders()          // refresca la lista

        await fetchAnalytics()        // refresca analíticas si las hay

        showMessage({

            message: `Order status updated`,

            type: 'success',

            style: GlobalStyles.flashStyle,

```

```
        titleStyle: GlobalStyles.flashTextStyle
    })
} catch (error) {
    showMessage({
        message: `Could not update status. ${error}`,
        type: 'error',
        style: GlobalStyles.flashStyle,
        titleStyle: GlobalStyles.flashTextStyle
    })
}
}
```

8. SECCIÓN DE ANALÍTICAS (Flex)

Estructura del objeto que devuelve el backend:

```
{
    "restaurantId": 1,
    "numYesterdayOrders": 2,
    "numPendingOrders": 1,
    "numDeliveredTodayOrders": 1,
    "invoicedToday": 65.0
}
```

Ejemplo de renderizado con los estilos proporcionados:

{analytics &&

```
<View style={styles.analyticsContainer}>
```

```
  <View style={styles.analyticsRow}>
```

```
    <View style={[styles.analyticsCell, { flex: 1 }]}>
```

```
      <TextSemiBold>Invoiced today</TextSemiBold>
```

```
      <TextRegular>{analytics.invoicedToday} €</TextRegular>
```

```
    </View>
```

```
    <View style={[styles.analyticsCell, { flex: 1 }]}>
```

```
      <TextSemiBold>Pending orders</TextSemiBold>
```

```
      <TextRegular>{analytics.numPendingOrders}</TextRegular>
```

```
    </View>
```

```
  </View>
```

```
<View style={styles.analyticsRow}>
```

```
  <View style={[styles.analyticsCell, { flex: 1 }]}>
```

```
    <TextSemiBold>Delivered today</TextSemiBold>
```

```
    <TextRegular>{analytics.numDeliveredTodayOrders}</TextRegular>
```

```
  </View>
```

```
  <View style={[styles.analyticsCell, { flex: 1 }]}>
```

```
    <TextSemiBold>Yesterday orders</TextSemiBold>
```

```
    <TextRegular>{analytics.numYesterdayOrders}</TextRegular>
```

```
  </View>
```

```

</View>

</View>

}

```

Propiedades Flex habituales en analytics:

```

analyticsContainer: { flexDirection: 'column', ... },

analyticsRow:      { flexDirection: 'row', justifyContent: 'space-between' },

analyticsCell:     { flex: 1, alignItems: 'center', padding: 10 }

```

9. GLOBALSTYLES — COLORES Y COMPONENTES VISUALES

Colores disponibles

Variable	Color	Uso típico
<code>brandPrimary</code>	 Granate <code>#be0f2e</code>	Botón Delete, acciones destructivas
<code>brandPrimaryTap</code>	 Granate oscuro <code>#AA001A</code>	Estado pressed de <code>brandPrimary</code>
<code>brandPrimaryDisabled</code>	 Granate transparente	Botón deshabilitado
<code>brandSecondary</code>	 Amarillo <code>#feca1b</code>	Destacados, badges
<code>brandSecondaryTap</code>	 Amarillo oscuro <code>#EAB607</code>	Estado pressed de <code>brandSecondary</code>
<code>brandSuccess</code>	 Verde <code>#95be05</code>	Botón Save/Confirm
<code>brandSuccessTap</code>	 Verde <code>#95be05</code>	Estado pressed de <code>brandSuccess</code>
<code>brandSuccessDisabled</code>	 Verde transparente	Save deshabilitado

Variable	Color	Uso típico
brandBlue	 Azul grisáceo #648a9f	Botón Edit, navegación
brandBlueTap	 Azul oscuro #527487	Estado pressed de brandBlue
brandGreen	 Verde azulado #059f94	Acción positiva alternativa
brandGreenTap	 Verde azulado oscuro #047f77	Estado pressed de brandGreen
brandBackground	 Gris claro	Fondo de pantallas

Pressable con color correcto según acción

// DELETE → granate

```
<Pressable onPress={() => handleDelete(item)}

  style={({ pressed }) => [

    { backgroundColor: pressed ? GlobalStyles.brandPrimaryTap : GlobalStyles.brandPrimary },

    styles.button

  ]}>

  <MaterialCommunityIcons name='delete' color='white' size={20} />

</Pressable>
```

// EDIT → azul

```
<Pressable onPress={() => navigation.navigate('EditScreen', { id: item.id })}

  style={({ pressed }) => [

    { backgroundColor: pressed ? GlobalStyles.brandBlueTap : GlobalStyles.brandBlue },

    styles.button

  ]}>

  <MaterialCommunityIcons name='pencil' color='white' size={20} />
```

```
</Pressable>
```

```
// SAVE → verde
```

```
<Pressable onPress={handleSubmit}
```

```
  style={({ pressed }) => [
```

```
    { backgroundColor: pressed ? GlobalStyles.brandSuccessTap : GlobalStyles.brandSuccess },
```

```
    styles.button
```

```
  ]}>
```

```
    <TextRegular textStyle={{ color: 'white' }}>Save</TextRegular>
```

```
</Pressable>
```

Iconos más usados (MaterialCommunityIcons)

Acción	name
Borrar	'delete'
Editar	'pencil'
Guardar	'content-save'
Avanzar estado	'skip-next'
Ver detalle	'eye'
Estrella llena	'star'
Estrella vacía	'star-outline'
Añadir	'plus-circle'
Volver	'arrow-left'

10. LAYOUTS VISUALES CON FLEX

Nombre + 1 botón en una línea (50/50)

```
<View style={{ flexDirection: 'row', alignItems: 'center', padding: 10 }}>

  <TextRegular style={{ flex: 1 }}>{item.name}</TextRegular>

  <Pressable onPress={() => handleDelete(item)}

    style={({ pressed }) => [

      { backgroundColor: pressed ? GlobalStyles.brandPrimaryTap : GlobalStyles.brandPrimary,
        flex: 1, padding: 10, alignItems: 'center' }

    ]}>

    <MaterialCommunityIcons name='delete' color='white' size={20} />

  </Pressable>

</View>
```

Nombre + 2 botones en una línea (nombre mitad, botones mitad)

```
<View style={{ flexDirection: 'row', alignItems: 'center', padding: 10 }}>

  {/* Mitad izquierda */}

  <TextRegular style={{ flex: 1 }}>{item.name}</TextRegular>

  {/* Mitad derecha: dos botones en fila */}

  <View style={{ flex: 1, flexDirection: 'row', justifyContent: 'flex-end', gap: 5 }}>

    <Pressable onPress={() => navigation.navigate('EditScreen', { id: item.id })}

      style={({ pressed }) => [

        { backgroundColor: pressed ? GlobalStyles.brandBlueTap : GlobalStyles.brandBlue,
          padding: 8, borderRadius: 5 }

      ]}

    </Pressable>

  </View>

</View>
```

```

    ]}>

    <MaterialCommunityIcons name='pencil' color='white' size={20} />

  </Pressable>

  <Pressable onPress={() => handleDelete(item)}

    style={({ pressed }) => [

      { backgroundColor: pressed ? GlobalStyles.brandPrimaryTap : GlobalStyles.brandPrimary,
padding: 8, borderRadius: 5 }

    ]}>

    <MaterialCommunityIcons name='delete' color='white' size={20} />

  </Pressable>

</View>

</View>

```

3 columnas en una línea — última con estrella + botón

```

<View style={{ flexDirection: 'row', alignItems: 'center', padding: 10 }}>

  /* Columna 1 */

  <TextRegular style={{ flex: 1 }}>{item.name}</TextRegular>

  /* Columna 2 */

  <TextRegular style={{ flex: 1, textAlign: 'center' }}>{item.description}</TextRegular>

  /* Columna 3: estrella de default + delete */

  <View style={{ flex: 1, flexDirection: 'row', justifyContent: 'flex-end', gap: 5 }}>

    <Pressable onPress={() => handleSetDefault(item)}>

      <MaterialCommunityIcons

```

```

        name={item.isDefault ? 'star' : 'star-outline'}

        color={item.isDefault ? GlobalStyles.brandSecondary : 'grey'}

        size={24}

    />

</Pressable>

<Pressable onPress={() => handleDelete(item)}

    style={({ pressed }) => [

        { backgroundColor: pressed ? GlobalStyles.brandPrimaryTap : GlobalStyles.brandPrimary,
padding: 8, borderRadius: 5 }

    ]}>

    <MaterialCommunityIcons name='delete' color='white' size={20} />

</Pressable>

</View>

</View>

```

Referencia de props de View

flexDirection — en qué dirección van los hijos:

flexDirection: 'row' // hijos en horizontal →

flexDirection: 'column' // hijos en vertical ↓ (valor por defecto)

flex — cuánto espacio ocupa este elemento (reparto proporcional entre hermanos):

// 50/50:

```
<View style={{ flex: 1 }} />
```

```
<View style={{ flex: 1 }} />
```

// 1/3 y 2/3:

```
<View style={{ flex: 1 }} />
```

```
<View style={{ flex: 2 }} />
```

justifyContent — alineación en el eje PRINCIPAL (horizontal si row, vertical si column):

justifyContent: 'flex-start' // agrupa al inicio (por defecto)

justifyContent: 'flex-end' // agrupa al final

justifyContent: 'center' // centra

justifyContent: 'space-between' // primero y último en extremos, resto distribuido

justifyContent: 'space-around' // espacio igual alrededor de cada hijo

justifyContent: 'space-evenly' // espacio exactamente igual entre todos

alignItems — alineación en el eje SECUNDARIO (el contrario al de flexDirection):

alignItems: 'flex-start' // alinea al inicio

alignItems: 'flex-end' // alinea al final

alignItems: 'center' // centra ← el más usado

alignItems: 'stretch' // estira para ocupar todo (por defecto)

padding y margin:

padding: 10 // espacio interior por los 4 lados

paddingHorizontal: 10 // solo izquierda y derecha

paddingVertical: 10 // solo arriba y abajo

margin: 10 // espacio exterior por los 4 lados

marginHorizontal: 10

marginVertical: 10

// padding = dentro del borde | margin = fuera del borde (separa del vecino)

11. BOTÓN REUTILIZABLE (Pressable estándar)

```
<Pressable

  onPress={() => navigation.navigate('OrdersScreen', { id: item.id })}

  style={{ pressed }} => [

    { backgroundColor: pressed ? GlobalStyles.brandBlueTap : GlobalStyles.brandBlue },

    styles.actionButton

  ]
>

<View style={{ flex: 1, flexDirection: 'row', justifyContent: 'center' }}>

  <MaterialCommunityIcons name='eye' color={'white'} size={20} />

  <TextRegular textStyle={styles.text}>Check orders</TextRegular>

</View>

</Pressable>
```

10. REGLAS YUP MÁS USADAS

```
yup.string().required('Campo obligatorio')

yup.string().max(255, 'Texto demasiado largo')

yup.string().nullable().url('URL no válida')

yup.string().nullable().email('Email no válido')
```

```
yup.number().required('Campo obligatorio')
```

```
yup.number().positive('Debe ser mayor que 0')
```

```
yup.number().integer('Debe ser entero')
```

```
yup.number().nullable()
```

11. IMPORTS MÁS FRECUENTES

```
import { useState, useEffect, useContext } from 'react'
```

```
import { View, FlatList, Pressable, ScrollView, Image, StyleSheet } from 'react-native'
```

```
import { MaterialCommunityIcons } from '@expo/vector-icons'
```

```
import { showMessage } from 'react-native-flash-message'
```

```
import { Formik, ErrorMessage } from 'formik'
```

```
import * as yup from 'yup'
```

```
import { AuthorizationContext } from '../../context/AuthorizationContext'
```

```
import TextRegular from '../../components/TextRegular'
```

```
import TextSemiBold from '../../components/TextSemiBold'
```

```
import TextError from '../../components/TextError'
```

```
import InputItem from '../../components/InputItem'
```

```
import * as GlobalStyles from '../../styles/GlobalStyles'
```

12. PATRONES RECURRENTES EN EXÁMENES PASADOS

Basado en los exámenes de curso y julio de 2024 y 2025 (Orders, Schedules, Address):

Lo que SIEMPRE piden:

1. **Listado de entidades** asociadas a un restaurante → FlatList + useEffect + fetch
2. **Editar una entidad** → Formik + Yup + initialValues cargados del backend
3. **Acción rápida con PATCH** → cambiar estado (nextStatus, toggle, activar/desactivar)
4. **Sección de estadísticas/analíticas** → datos del backend mostrados en grid con Flex
5. **Botón condicional** → mostrar/ocultar según estado o condición del ítem

Lo que PUEDE aparecer pero no siempre:

- Crear entidad nueva (POST) → igual que editar pero initialValues vacíos
- Eliminar con confirmación → `Alert.alert` antes del `destroy`
- Filtrar/ordenar el listado en frontend (sin llamada extra al backend)
- Mostrar imagen según un campo (status, tipo, etc.)

Patrón DELETE con confirmación (en el return final):

```
<DeleteModal
  isVisible={addressToBeDeleted !== null}
  onCancel={() => setAddressToBeDeleted(null)}
  onConfirm={() => {
    removeAddress(addressToBeDeleted)
  }}
>
  <TextRegular>¿Seguro que quieres eliminar esta dirección?</TextRegular>
</DeleteModal>
```

Patrón imagen según estado:

// Función helper dentro del componente o en un archivo utils

```
const getImageByStatus = (status) => {

  if (status === 'pending') return require('../assets/pending.png')

  if (status === 'in process') return require('../assets/inProcess.png')

  if (status === 'sent') return require('../assets/sent.png')

  if (status === 'delivered') return require('../assets/delivered.png')

  return require('../assets/generic.png')
```

```
}
```

// Uso en el renderItem:

```
<Image source={getImageByStatus(item.status)} style={styles.image} />
```

Orden de ataque recomendado:

1. Stack → registrar pantallas nuevas (5 min, no falles aquí)
2. Endpoint → crear las funciones de API que necesites (10 min)
3. RF de LISTADO → es el más directo, te da confianza (15-20 min)
4. RF de PATCH/acción rápida → también sencillo (10-15 min)
5. RF de EDIT/CREATE → Formik, el más largo (20-25 min)
6. RF de ANALÍTICAS → si queda tiempo y puntos (10-15 min)

Errores típicos que cuestan puntos:

- ❌ Olvidar registrar la pantalla nueva en el Stack → navegación rota
- ❌ `initialValues` no coincide con el nombre del campo del backend → Formik no precarga
- ❌ No poner `enableReinitialize` en Formik de edición → los campos aparecen vacíos
- ❌ No poner `await` en los fetch tras una acción → datos desactualizados en pantalla
- ❌ Pasar mal los parámetros en `navigation.navigate` → pantalla destino sin datos
- ❌ No manejar el catch en los fetch → app se rompe sin mensaje de error
- ❌ Olvidar `[route]` en el array de dependencias del `useEffect` → no refresca al volver
- ❌ No importar `patch` de `ApiRequestsHelper` cuando usas PATCH

Cómo leer las rutas del backend rápido:

El enunciado o el README te dirá algo como:

GET `/restaurants/:restaurantId/schedules` → `getSchedules(restaurantId)`

POST `/schedules` → `create(data)`

PUT /schedules/:id → update(id, data)

DELETE /schedules/:id → remove(id)

PATCH /schedules/:id/toggle → toggle(id)

Mapea directamente a las funciones de ApiRequestsHelper y ya tienes el Endpoint.

Si te atascas en algo:

- Deja un comentario `// TODO` y pasa al siguiente RF
- Un RF sin terminar al 50% puede dar puntos parciales; uno sin empezar da 0
- Lo visual (estilos, flex, colores) va después de que funcione la lógica

14. CHECKLIST DE ENTREGA

- ☐ Backend arranca sin errores (`npm run start:backend`)
- ☐ Frontend arranca sin errores (`npm run start:frontend`)
- ☐ Todas las pantallas nuevas registradas en el Stack
- ☐ Todos los RFs del enunciado implementados (o con avance parcial)
- ☐ Validaciones Yup completas y mensajes de error visibles
- ☐ Los fetch usan `await` correctamente tras acciones
- ☐ Borrar carpetas `node_modules` y `.expo` antes de comprimir
- ☐ Comprobar que el ZIP incluye la solución (no es el original vacío)
- ☐ Esperar enlace de confirmación en la plataforma antes de enviar

IMPORTANTES LOS ENDPOINTS Y LOS STACK NADA MAS EMPEZAR EL EXAMEN

Clase Create: -InitialValues

-Validation

-Create

-Return: Formik

ej:

```
return (  
  <Formik  
    validationSchema={validationSchema}  
    initialValues={initialProductValues}  
    onSubmit={createProduct}>  
    {{{ handleSubmit, setFieldValue, values }}} => (  
      <div>  
        <div>  
          <div>  
            <div>  
              <div>  
                <div>  
                  <div>  
                    <div>  
                      <div>  
                        <div>  
                          <div>  
                        </div>  
                      </div>  
                    </div>  
                  </div>  
                </div>  
              </div>  
            </div>  
          </div>  
        </div>  
      </div>  
    )  
  )  
)
```

```

<ScrollView>
  <View style={{ alignItems: 'center' }}>
    <View style={{ width: '60%' }}>
      <FormItem
        name='name'
        label='Name:'
      />

      {backendErrors &&
        backendErrors.map((error, index) => <TextError
key={index}>{error.param}-{error.msg}</TextError>
        )
      }

      <Pressable
        onPress={handleSubmit}
        style={({ pressed }) => [
          {
            backgroundColor: pressed
              ? GlobalStyles.brandSuccessTap
              : GlobalStyles.brandSuccess
          },
          styles.button
        ]}>
        <View style={([ flex: 1, flexDirection: 'row', justifyContent:
'center' ])}>
          <MaterialCommunityIcons name='content-save' color={'white'}
size={20} />
          <TextRegular textStyle={styles.text}>
            Save
          </TextRegular>
        </View>
      </Pressable>
    </View>
  </ScrollView>
) }
</Formik>
)

```

Clase Principal: Screen:

- Estados
- UseEffect(ver si hay que esta logeado o no)

- fetch
 - RenderProduct(Pastelito bueno)
 - RenderHeader
 - RenderIfEmpty
 - Si hay remove se hace aqui
 - Return flatlist(Si hay remove, hay deleteModal para asegurarnos de que se borra)
- Ej:

```

return (
  <>
    <FlatList
      style={styles.container}
      data={categories}
      renderItem={renderCategory}
      keyExtractor={item => item.id.toString()}
      ListHeaderComponent={renderHeader}
      ListEmptyComponent={renderEmptyCategoriesList}
    />
    <DeleteModal
      isVisible={categoryToBeDeleted !== null}
      onCancel={() => setCategoryToBeDeleted(null)}
      onConfirm={() => removeProductCategory(categoryToBeDeleted)}>
      <TextRegular>If the category has products, it cannot be
deleted.</TextRegular>
    </DeleteModal>
  </>
)

```

En el caso de productCategory, hacer en CreateProductScreen y EditProductScreen el reshaped para que funcione el dropdownpicker

DropdDownPicker, se usa para cuando hay que elegir entre varias opciones:

ej:

```

<DropDownPicker
  open={open}
  value={values.productCategoryId}
  items={productCategories}
  setOpen={setOpen}
  onSelectItem={item => {

```

```

        setFieldValue('productId', item.value)
    }}
    setItems={setProductCategories}
    placeholder="Select the product category"
    containerStyle={{ height: 40, marginTop: 20, marginBottom:
20 }}

    style={{ backgroundColor: GlobalStyles.brandBackground }}
    dropDownStyle={{ backgroundColor: '#fafafa' }}
  />

```

EJEMPLO DE SWITCH:

```

<TextRegular>Is it available?</TextRegular>
    <Switch
        trackColor={{ false: GlobalStyles.brandSecondary, true:
GlobalStyles.brandPrimary }}
        thumbColor={values.availability ?
GlobalStyles.brandSecondary : '#f4f3f4'}
        // onChange={toggleSwitch}
        value={values.availability}
        style={styles.switch}
        onChange={value =>
            setFieldValue('availability', value)
        }
    />
    <ErrorMessage name={'availability'} render={msg =>
<TextError>{msg}</TextError> } />

```

EJEMPLO DE TOGGLE:

// "Toggle" implementado con Pressable

```

<Pressable onPress={() => handleToggleIsUnlimited(item)}>
    <MaterialCommunityIcons
        name={item.isUnlimited ? 'toggle-switch' : 'toggle-switch-off'}
        size={30}
        color={item.isUnlimited ? GlobalStyles.brandSuccess : 'grey'}
    />
</Pressable>

```

```
const unlimitedCount = restaurants.filter(r => r.isUnlimited).length
```